DevOps



LAB 06

Set Up Nexus Repository and Configure User Access

Submitted To: Sir Waqas Saleem

Submitted By: Ayesha Rehan

Registration No: 2023-BCS-021

Course Code: CS-649

GitHub Username: ayesha-rehan12

BCS 7-B

Department Of Computer Science

Task 1: Run Nexus Repository and Complete the Initial Setup

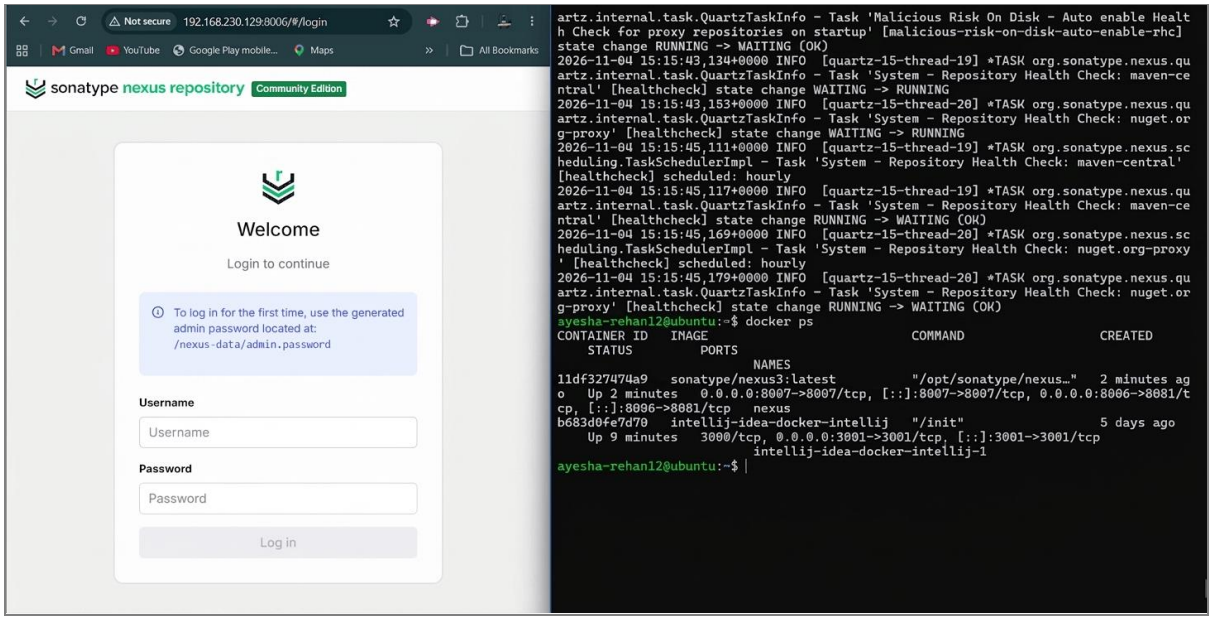


Figure: docker ps | grep nexus showing the running nexus container and its published ports 8006 and 8007.

Task 2: Create a Role and User and Verify Access

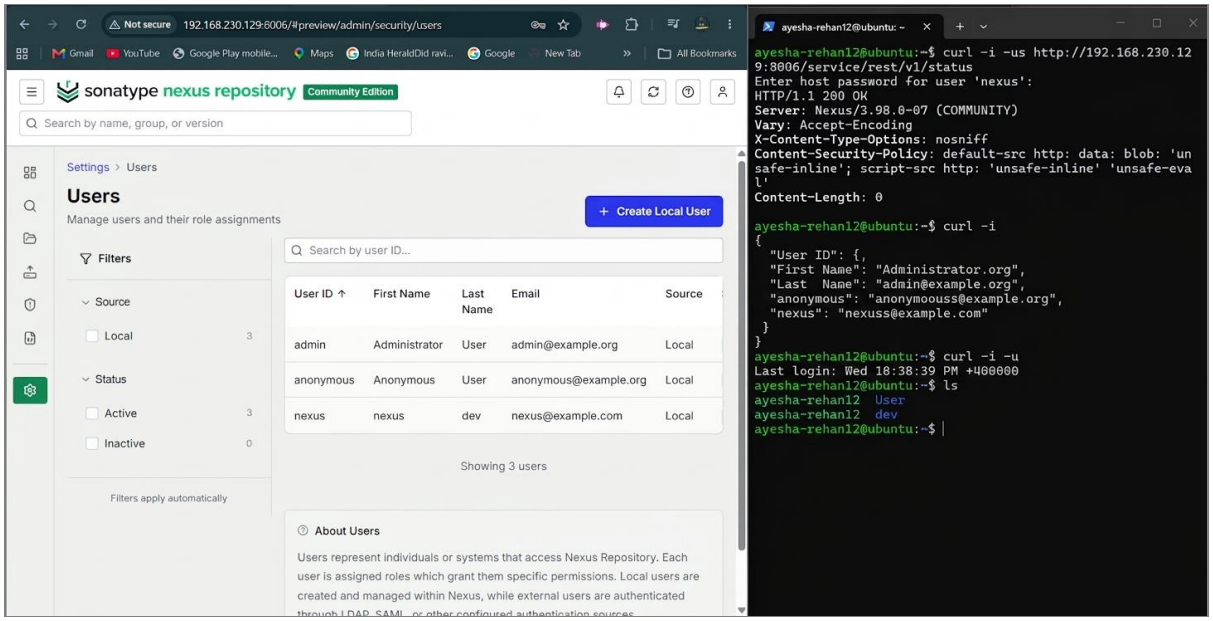


Figure: curl status API request as the nexus user returning HTTP/1.1 200 OK (password not shown).

Task 3: Build and Publish the Gradle Application to Nexus

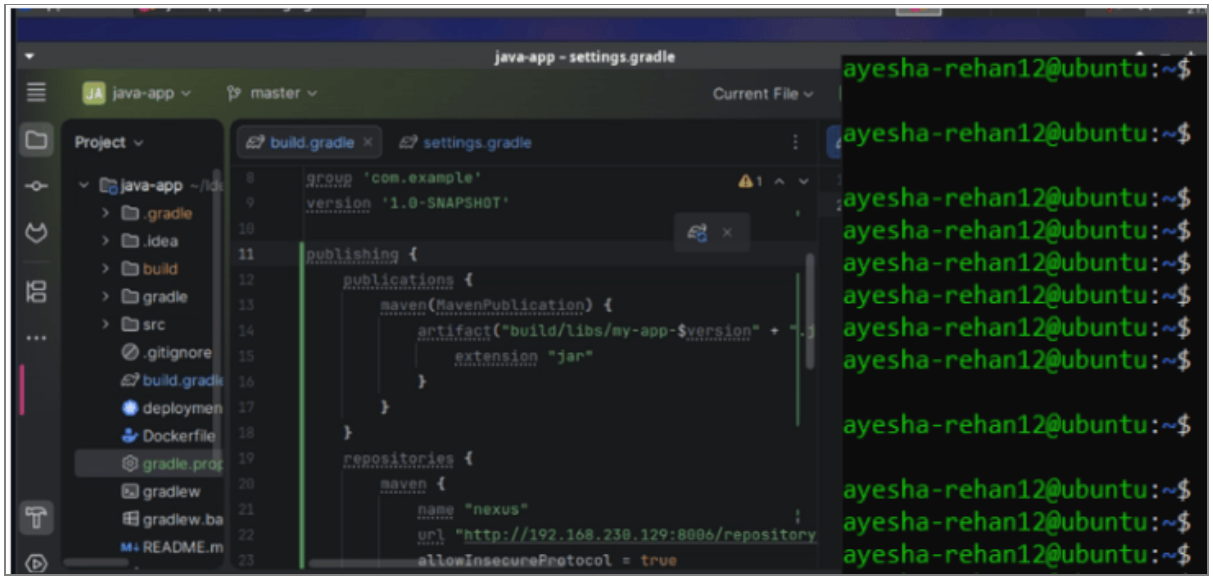


Figure: Updated build.gradle publishing configuration and settings.gradle (credentials not shown).

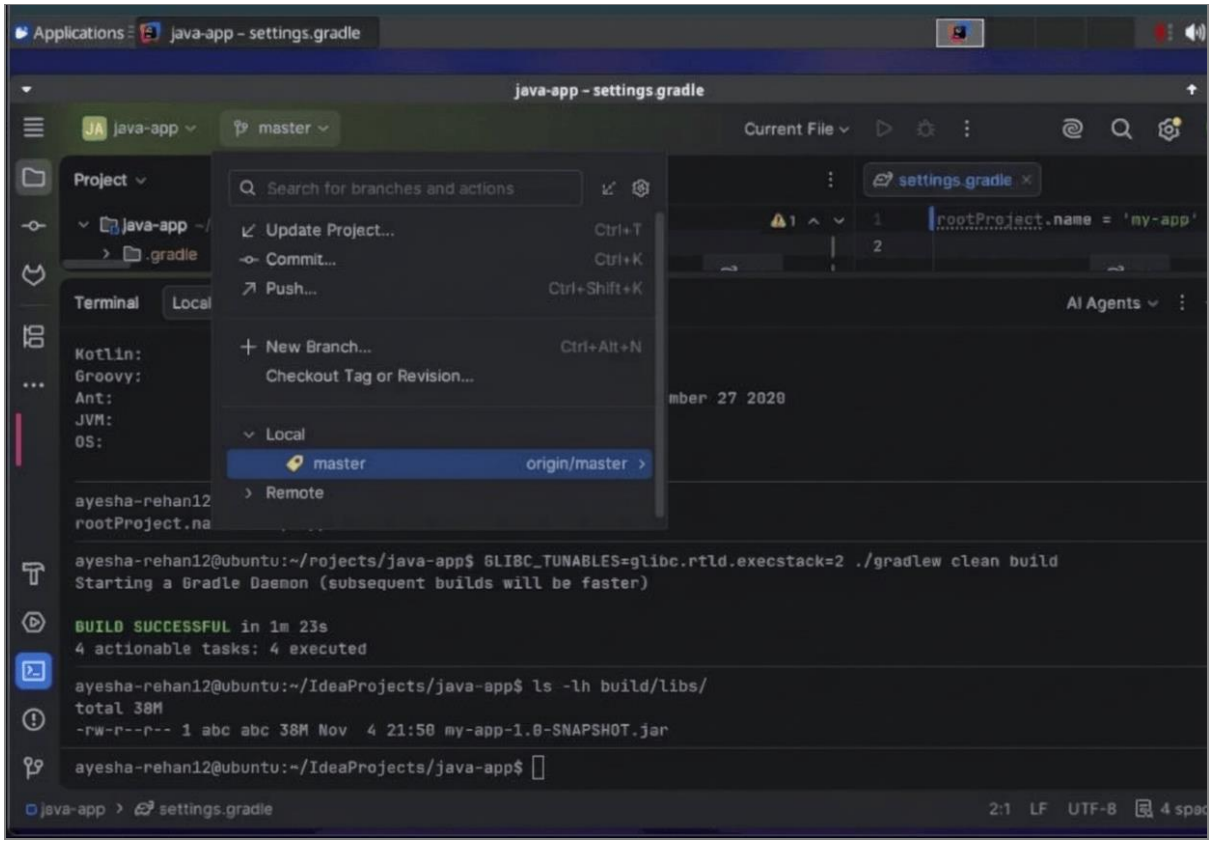


Figure: ./gradlew clean build with BUILD SUCCESSFUL and my-app-1.0-SNAPSHOT.jar generated.

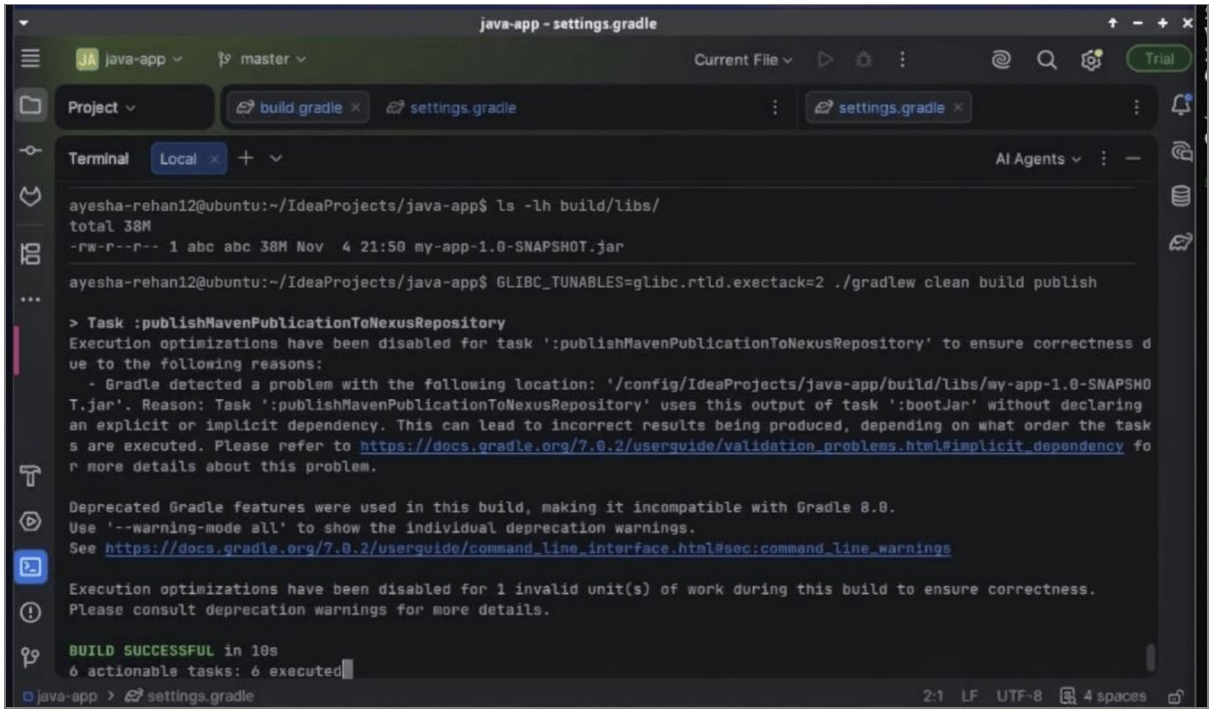


Figure: ./gradlew clean build publish running the Nexus publish task with BUILD SUCCESSFUL.

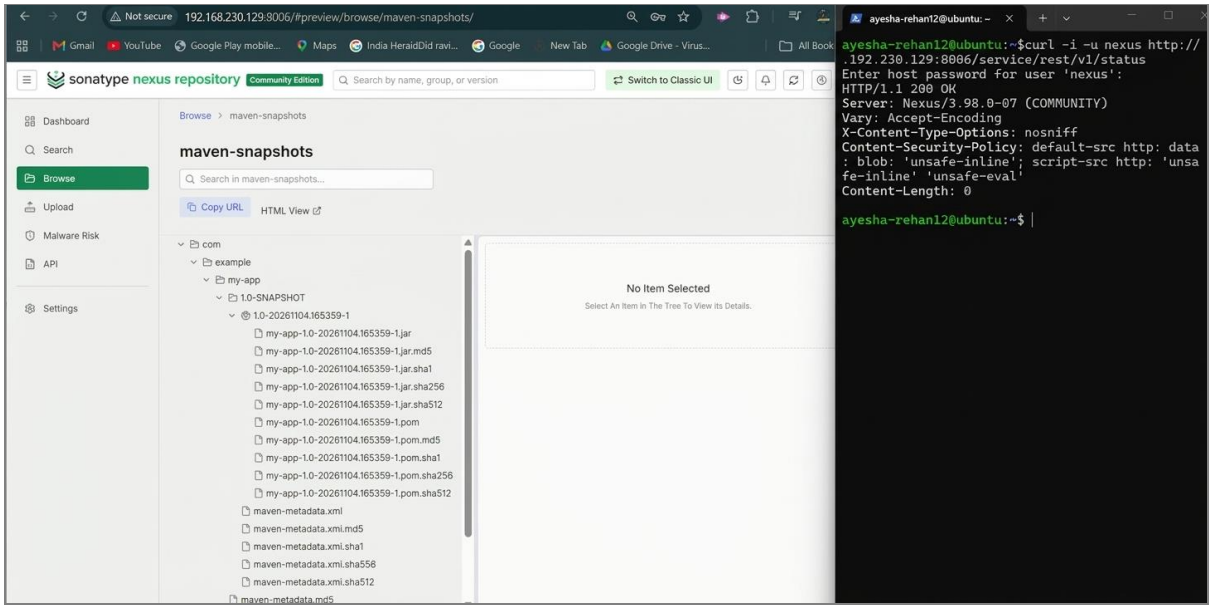


Figure: Published my-app snapshot JAR and metadata under com/example/my-app/1.0-SNAPSHOT in maven-snapshots.

Task 4: Publish the Maven Application to Nexus

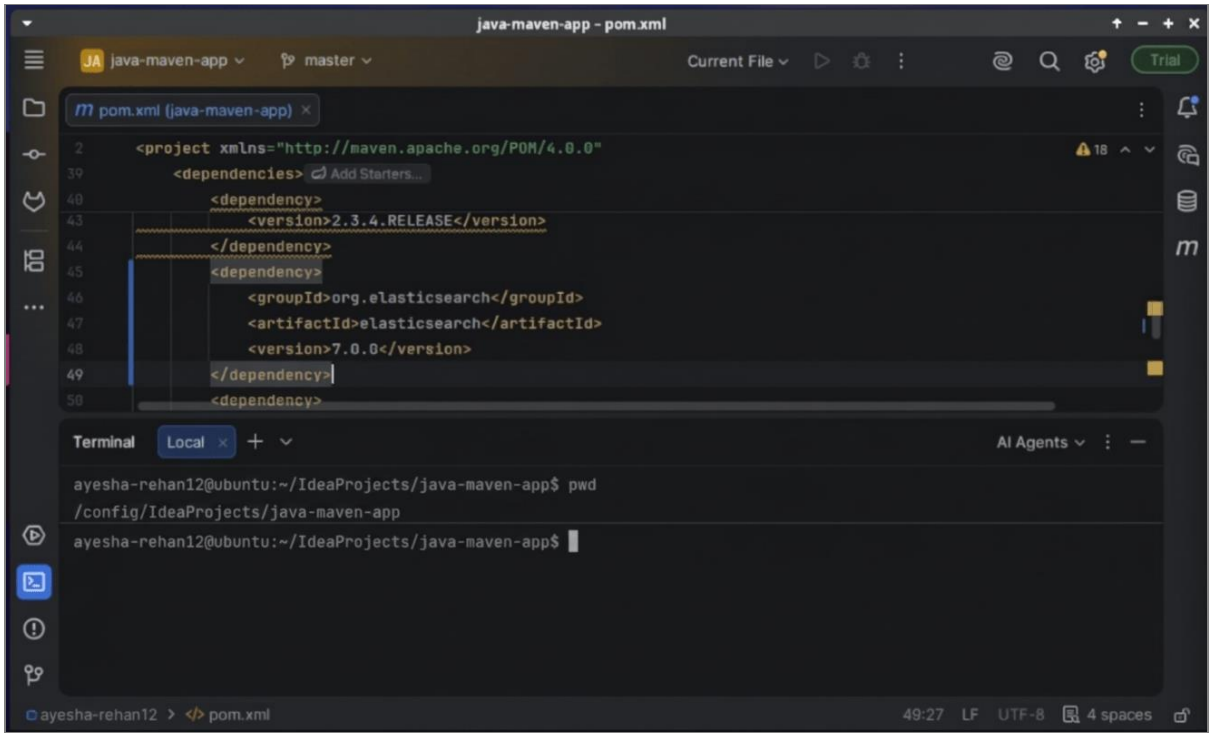


Figure: java-maven-app project opened successfully in IntelliJ.

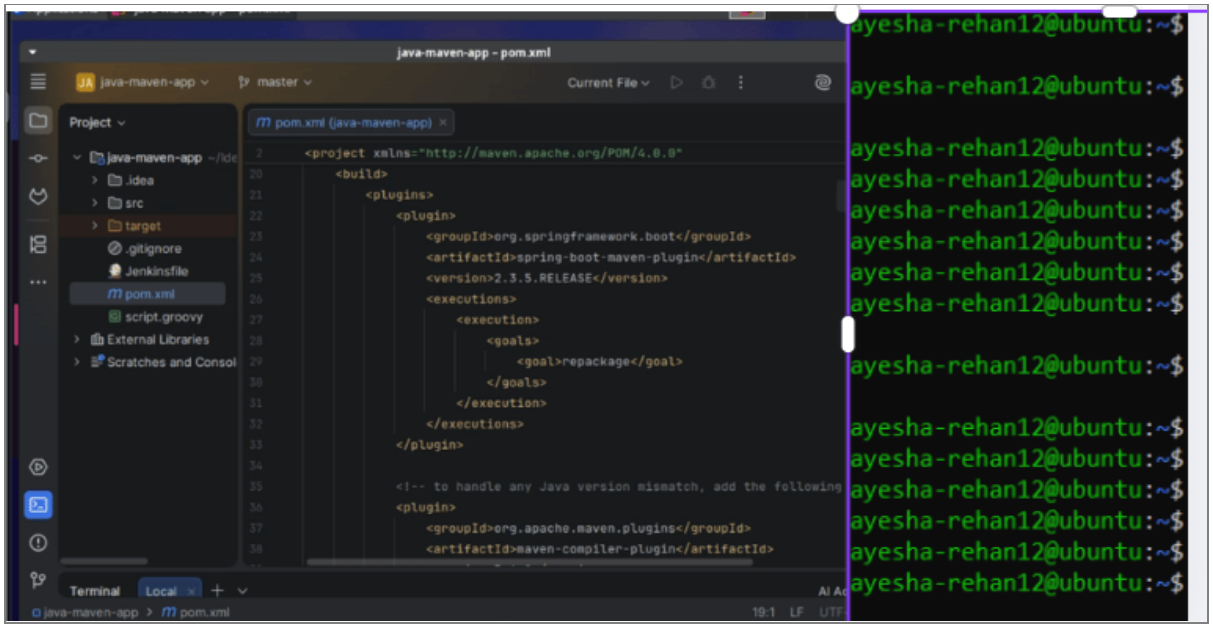


Figure: distributionManagement block added to pom.xml.

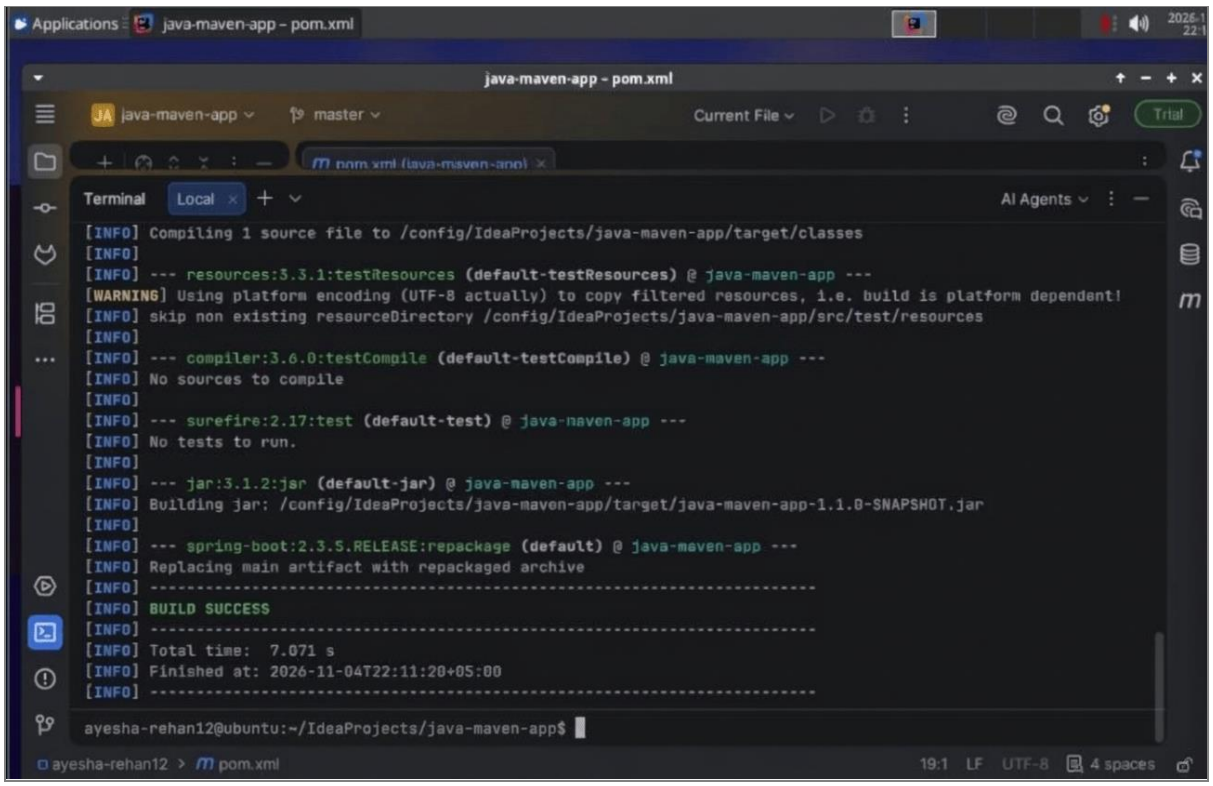


Figure: mvn package with BUILD SUCCESS and the generated snapshot JAR.

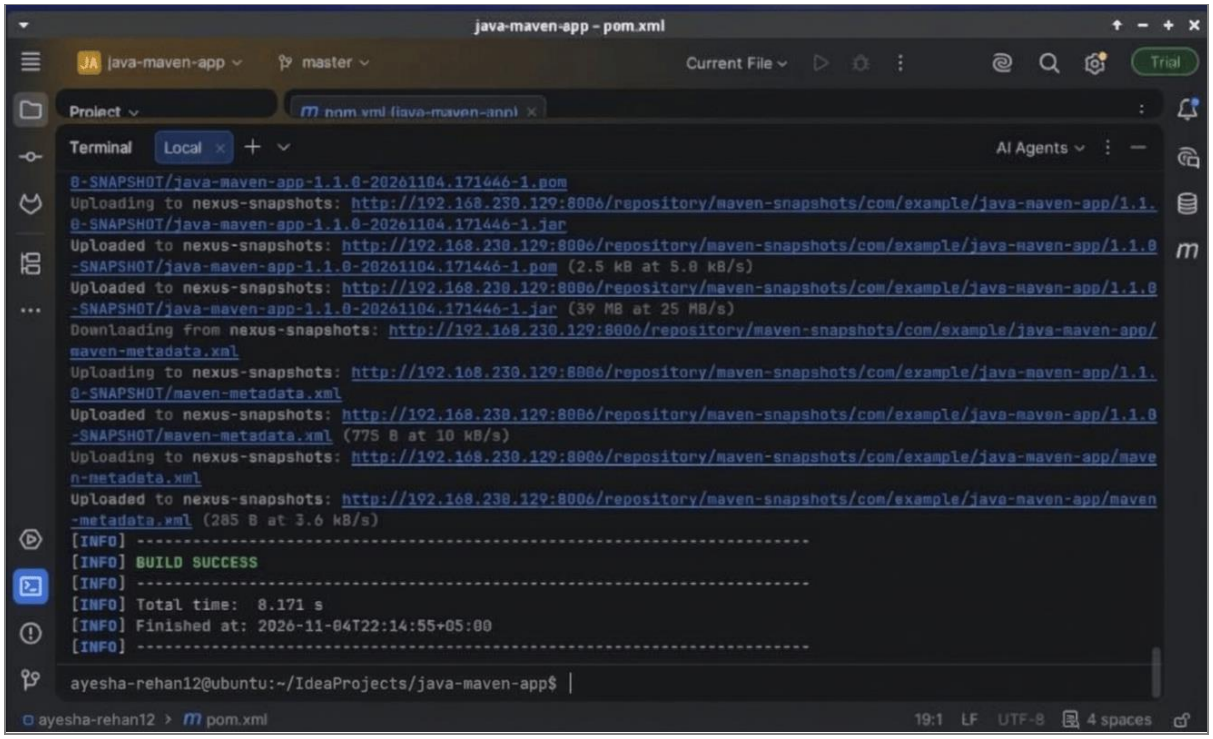


Figure: mvn deploy uploading artifacts to Nexus with BUILD SUCCESS (credentials not shown).

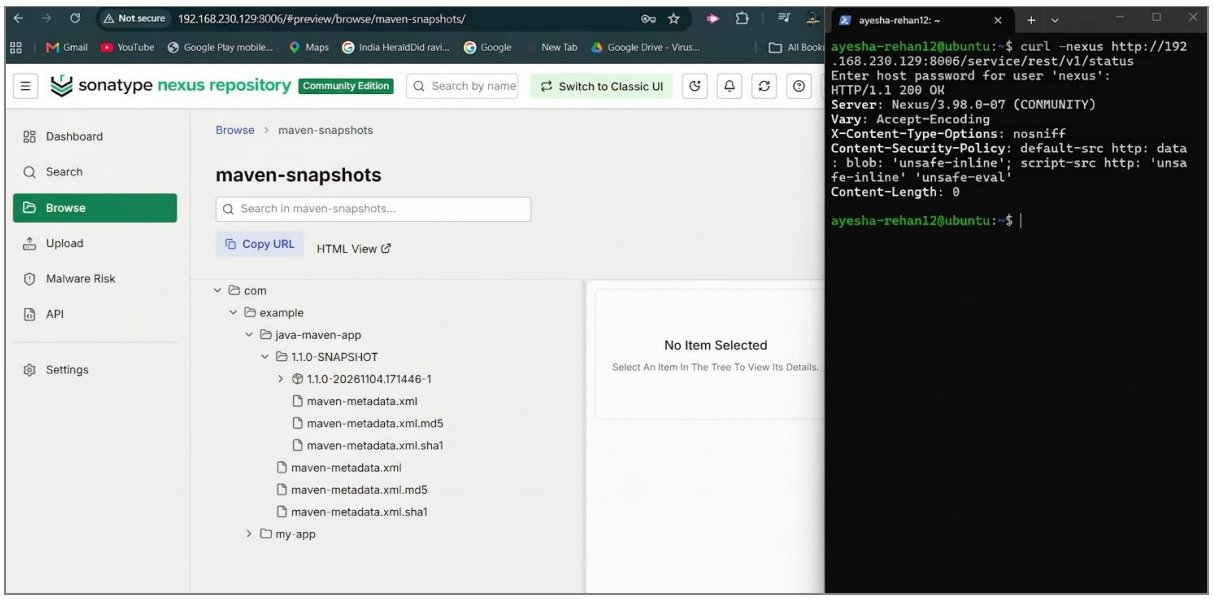


Figure: Published JAR, POM, and metadata under com/example/java-maven-app/1.1.0-SNAPSHOT.

Task 5: Inspect Nexus Repositories and Components Using the REST API

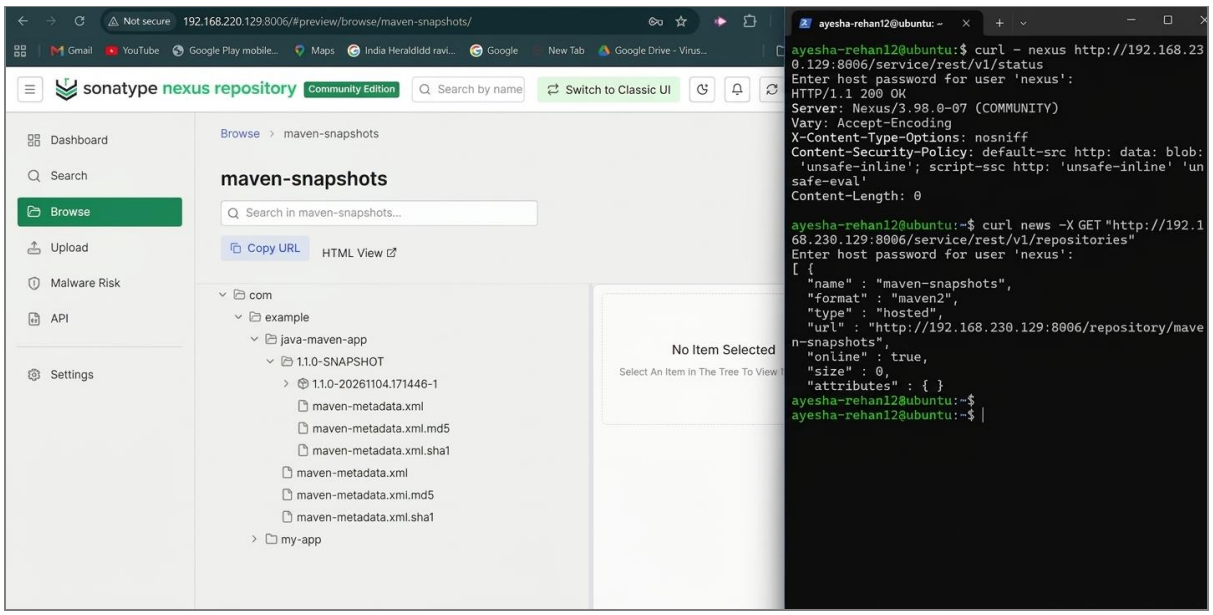


Figure: Repository list returned for the restricted nexus user.

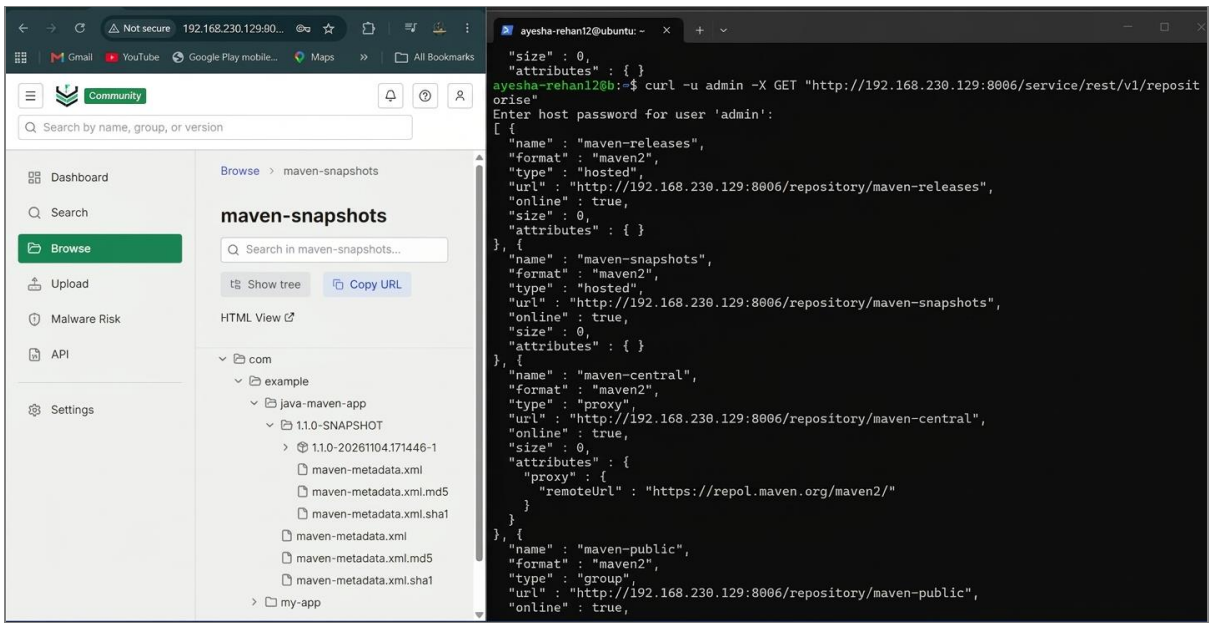


Figure: Repository list returned for the admin user.

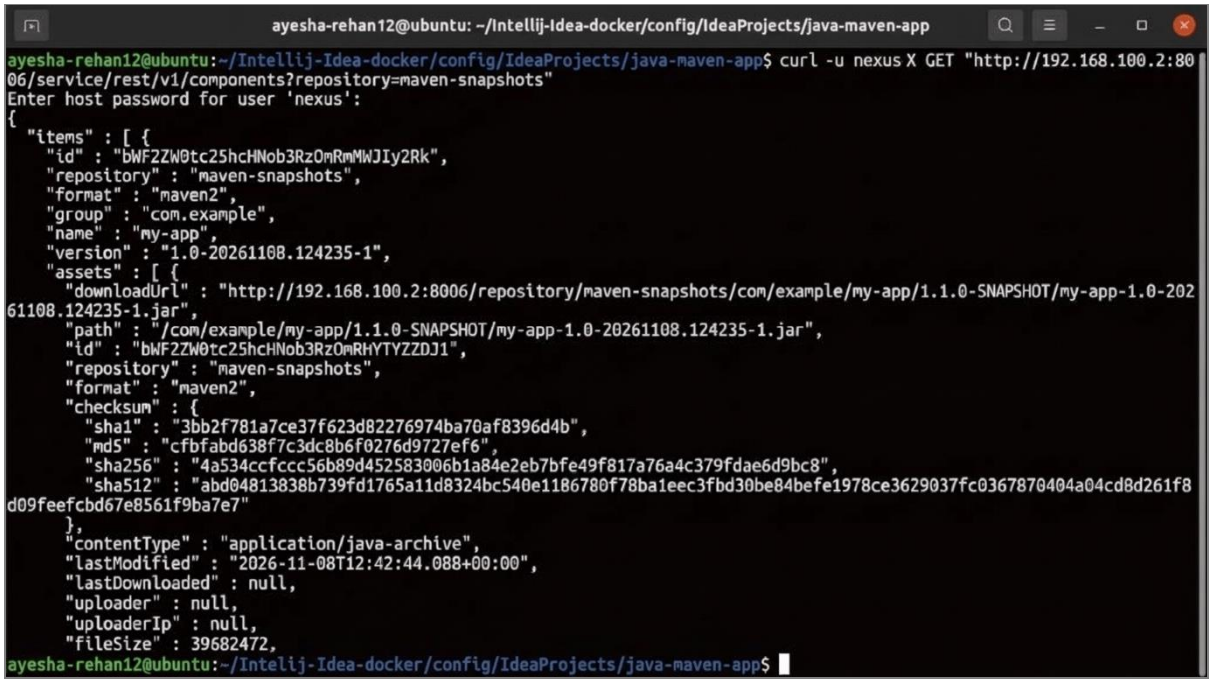


Figure: Components in maven-snapshots listed with id, repository, name, and version.

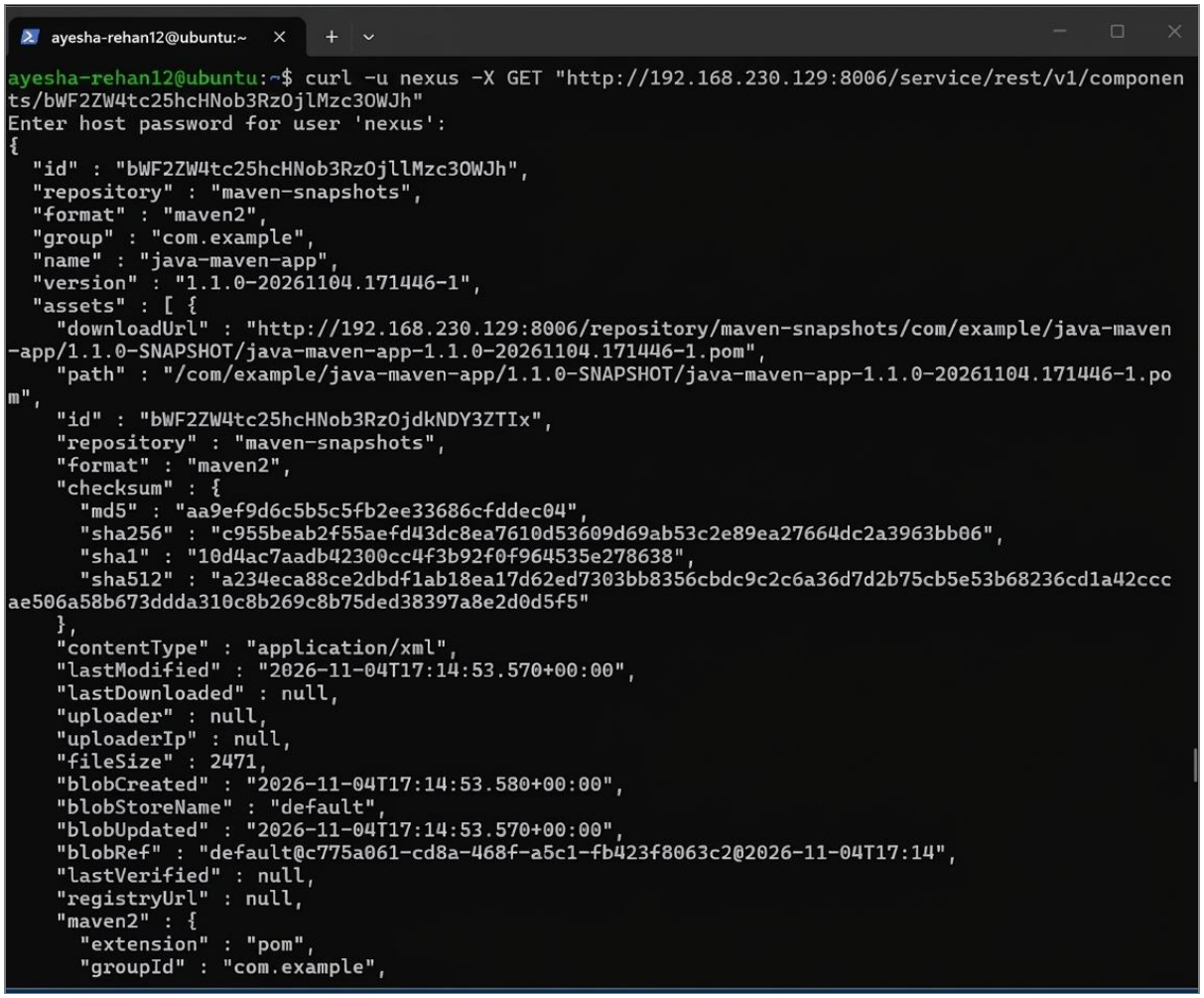


Figure: Details of one component retrieved by its ID.

Task 6: Create a Nexus Blob Store and Docker Hosted Repository

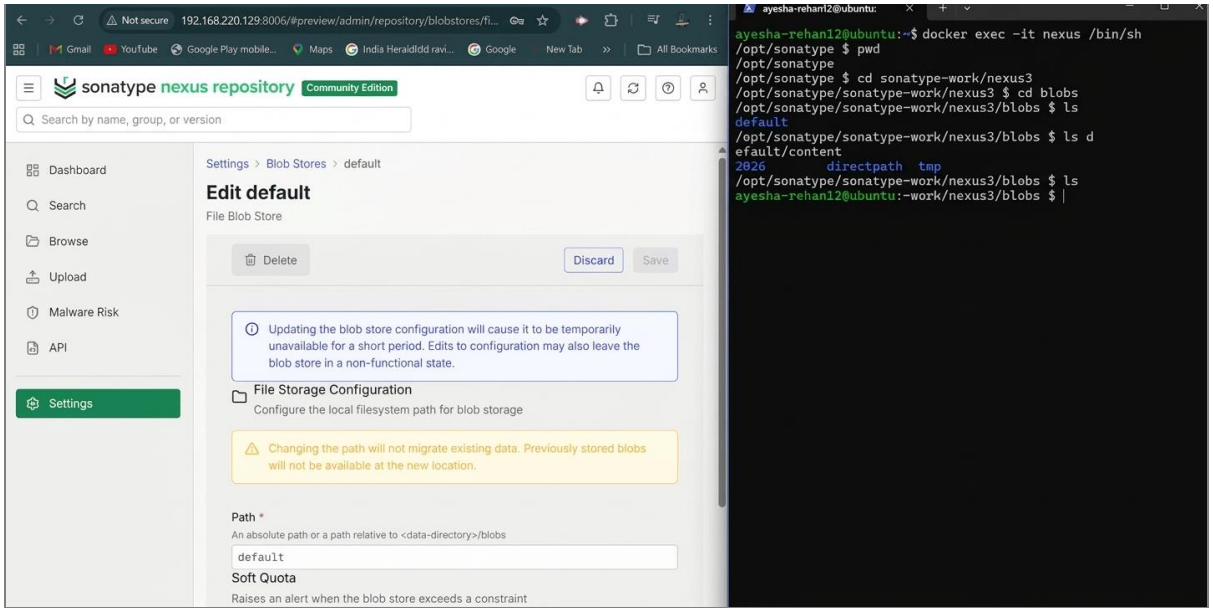


Figure: Default blob store directory inspected inside the Nexus container.

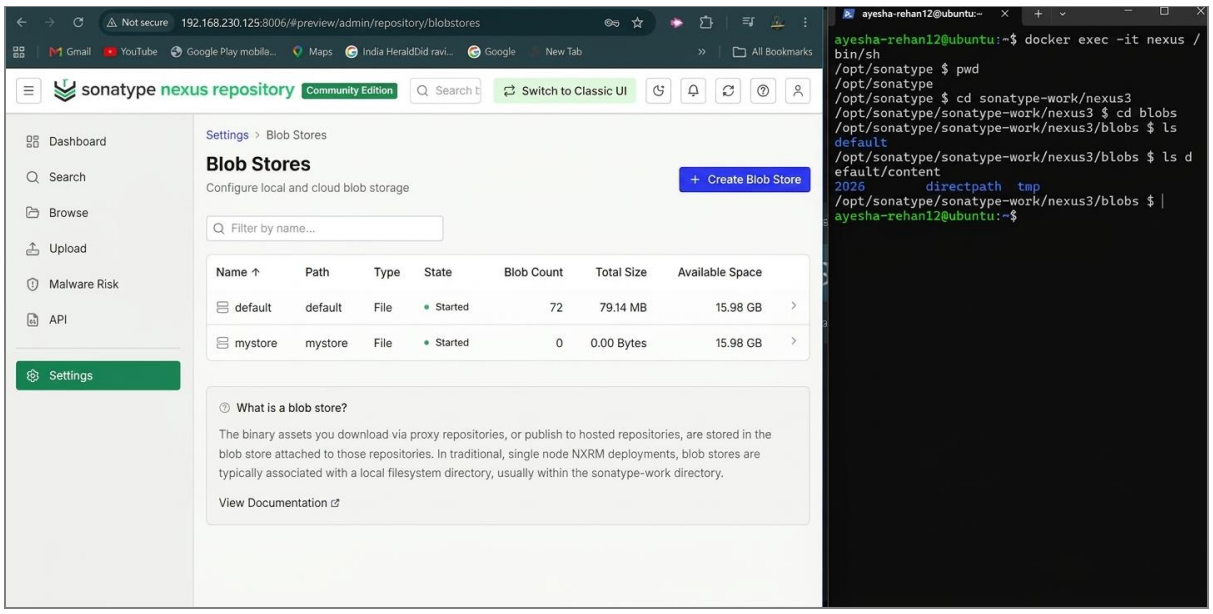


Figure: Blob Stores page showing default and mystore in the Started state.

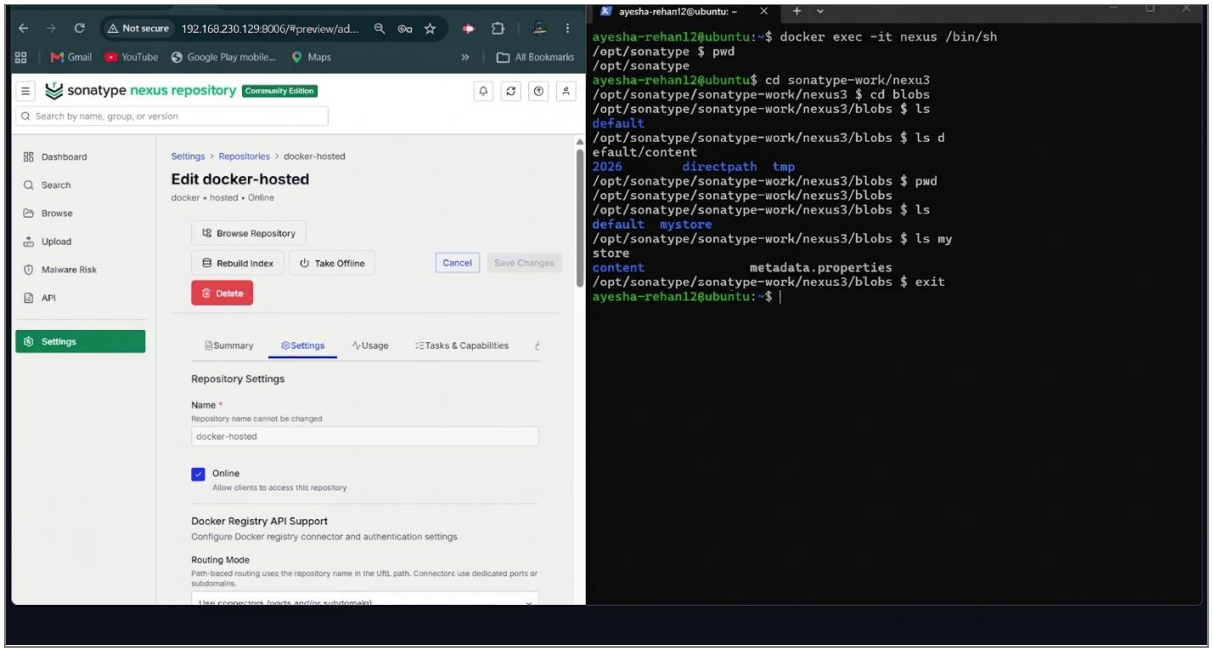


Figure: blobs directory inside the container showing default and mystore.

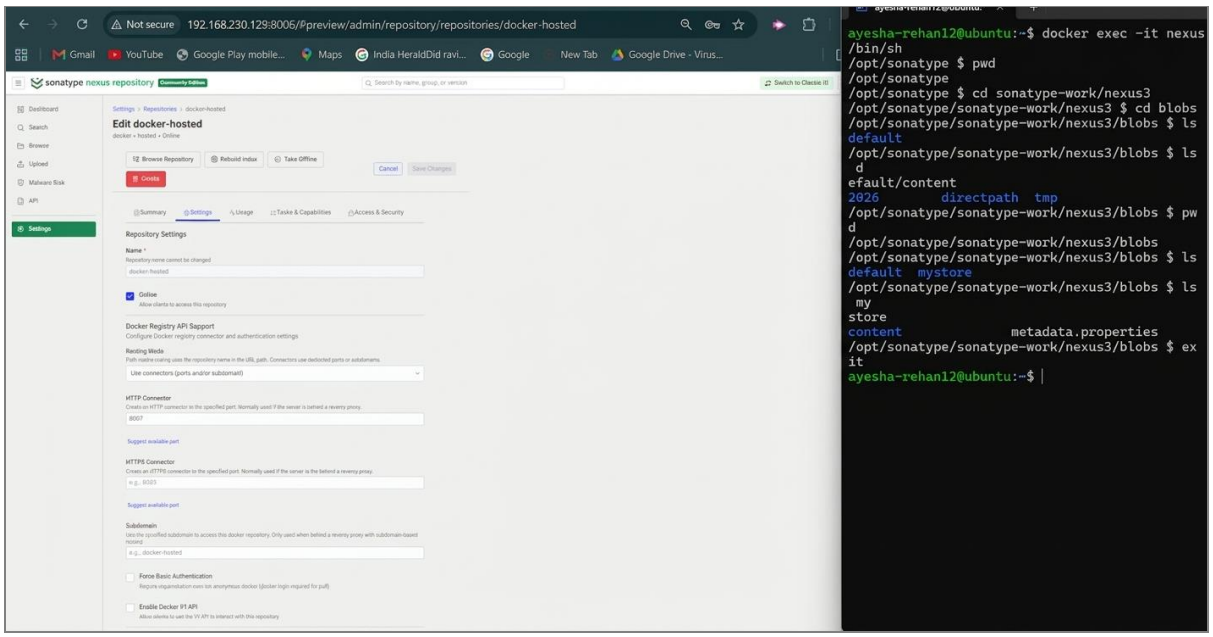


Figure: docker-hosted repository using the mystore blob store with HTTP connector 8007.

Task 7: Create and Run a Maven Snapshot Cleanup Policy

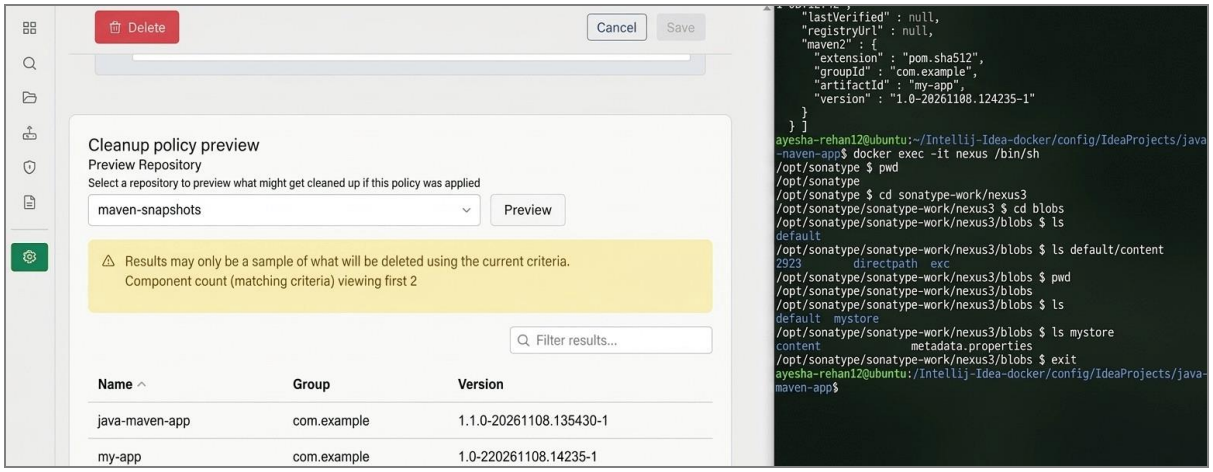


Figure: maven cleanup policy created with maven2 format, snapshot release type, . matcher, and preview.*

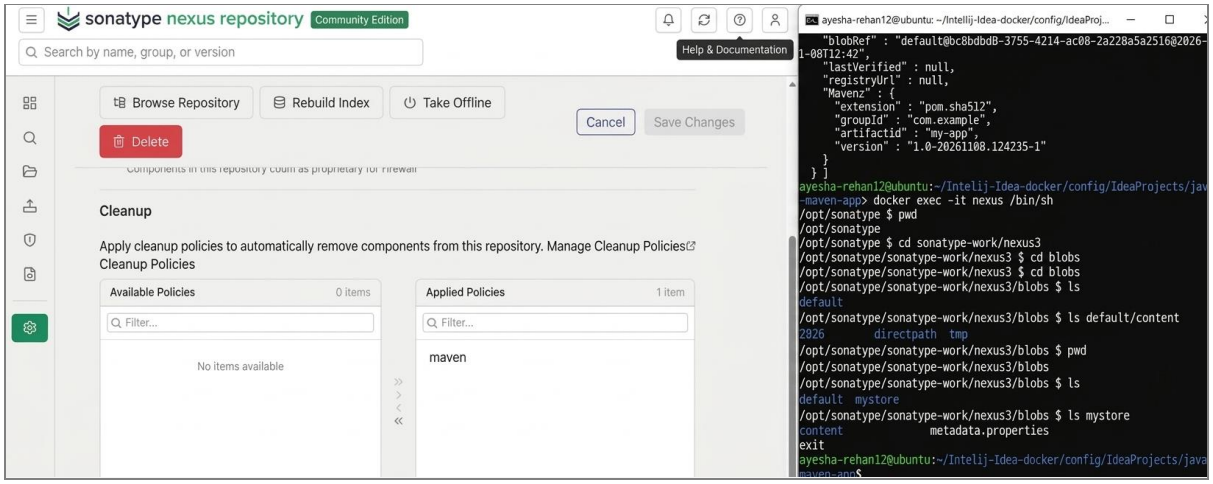


Figure: maven cleanup policy applied to the maven-snapshots repository.

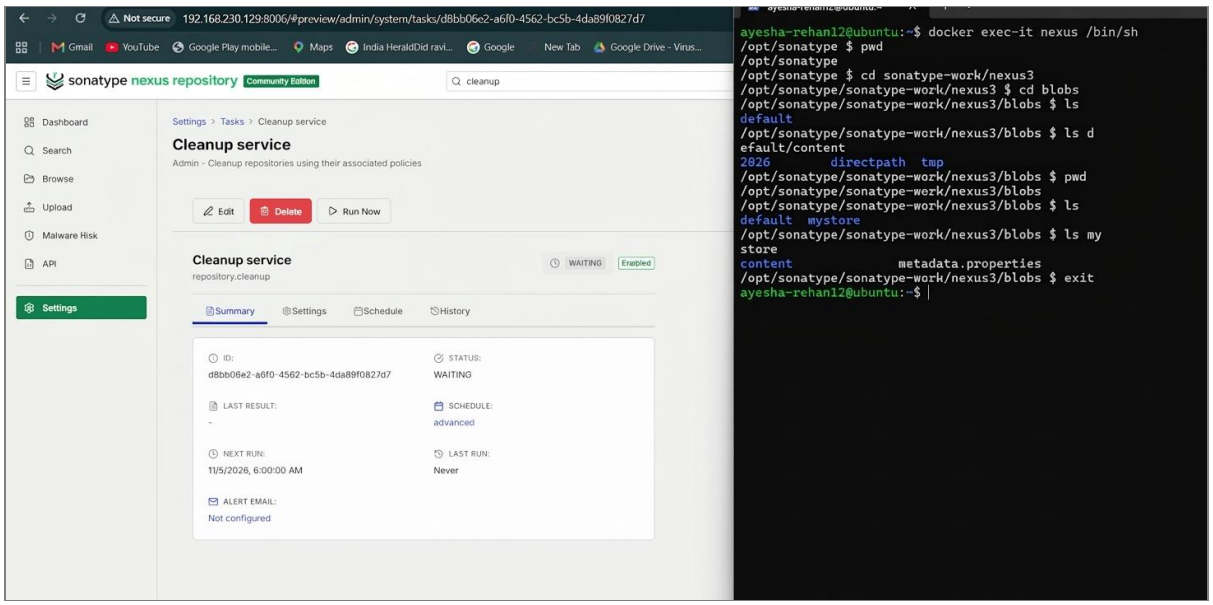


Figure: Cleanup service task in Waiting state, scheduled for 6:00 AM.

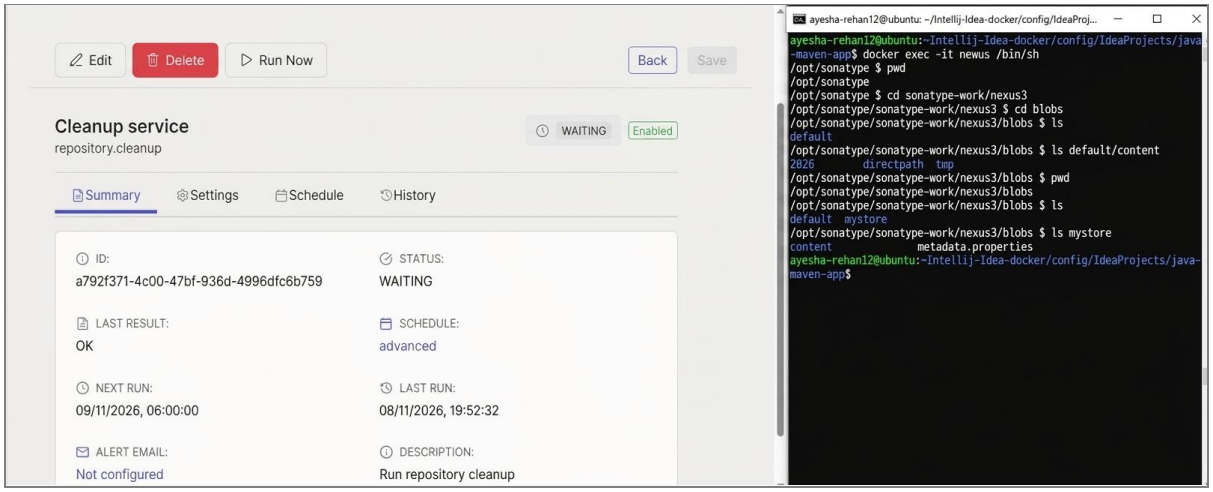


Figure: Cleanup service after manual run with updated Last run and Last result: Ok.

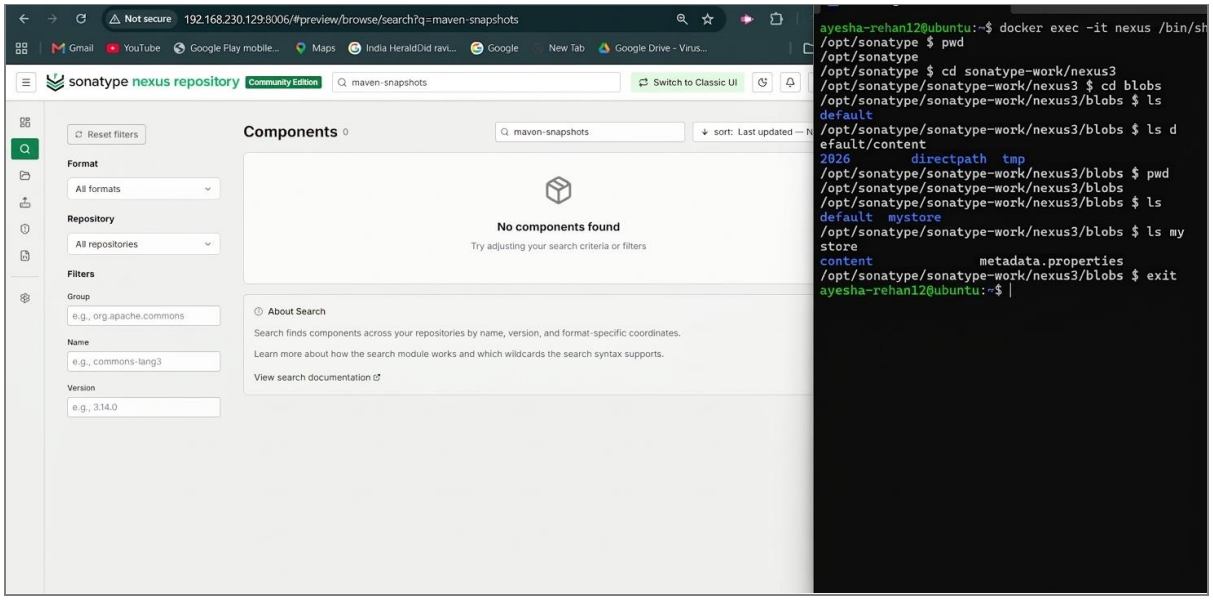


Figure: maven-snapshots showing 'No component/assets found in repository'.

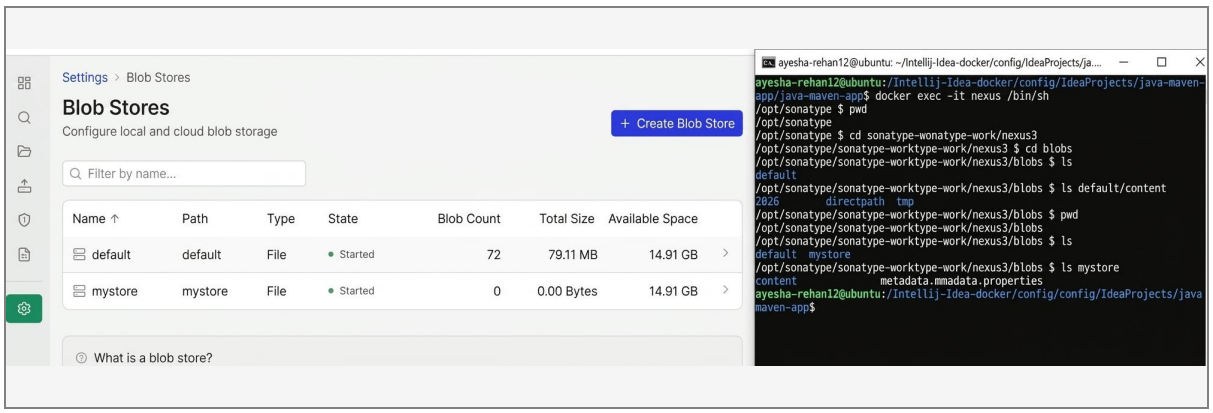


Figure: Default blob store blob count, used size, and available space before compaction.

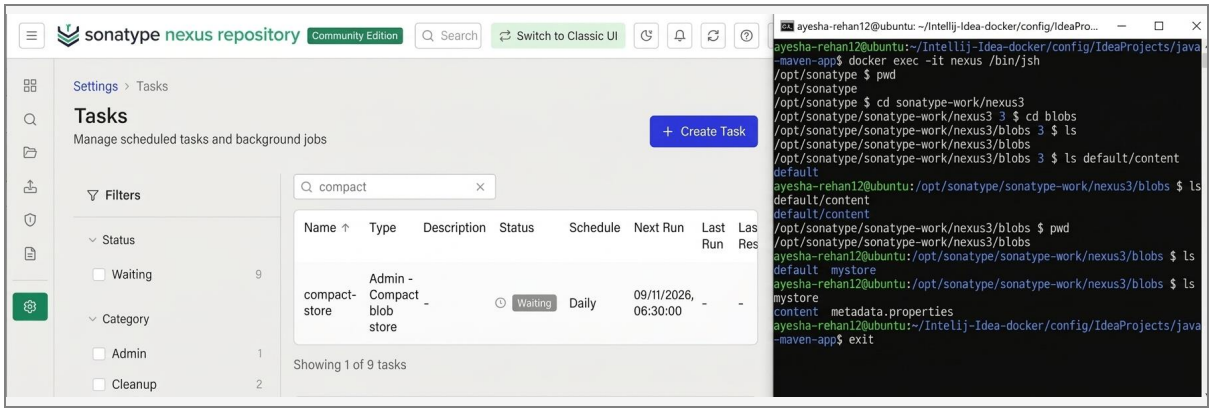


Figure: compact-store task configured for the default blob store, 0 days, daily at 06:30.

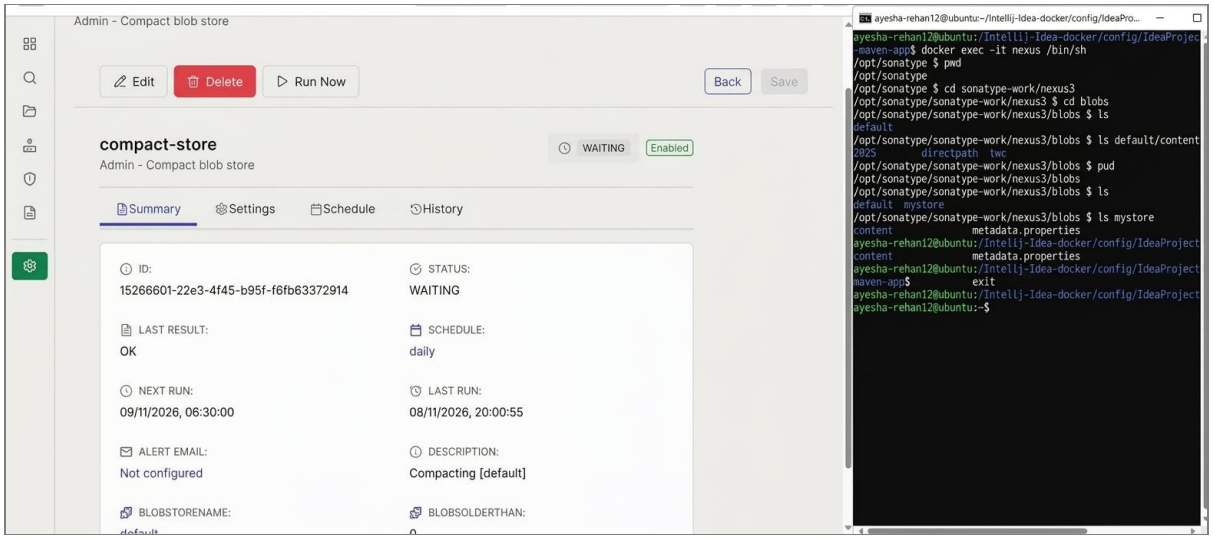


Figure: compact-store task after manual run with Last result: Ok.

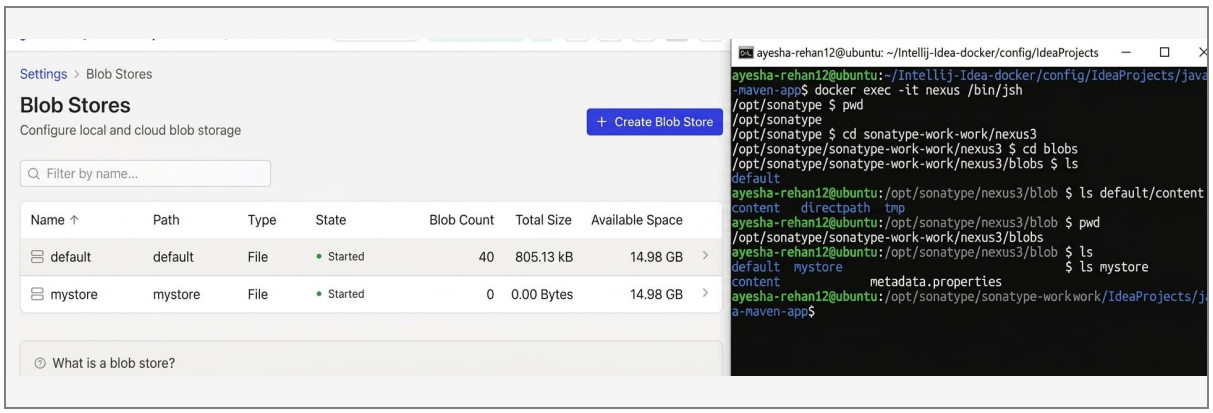


Figure: Default blob store after compaction with reduced used size and increased available space.